

# **A COMPREHENSIVE STUDY OF MACHINE LEARNING MODELS FOR CROSS-SITE SCRIPTING DETECTION**

## **RESEARCH QUESTION**

### **1. Describe the research question**

How can machine learning models be effectively utilized to detect Cross-Site Scripting (XSS) vulnerabilities in web applications?

Which machine learning models—CNN, RNN, or Transformers—are most effective in detecting and classifying XSS attacks?

XSS attacks inject malicious scripts into web pages, leading to unauthorized actions and data breaches; this research aims to address these threats by applying machine learning algorithms to classify web-based code inputs as benign or XSS-affected.

### **2. What were you trying to find in the dataset?**

The aim of this research is to determine how effectively machine learning models can detect Cross-Site Scripting (XSS) vulnerabilities in web applications. Specifically, the focus is on identifying and classifying web-based code snippets as either benign or malicious, providing a robust mechanism to enhance web security.

### **3. Why is this problem important?**

Cross-Site Scripting (XSS) attacks threaten web applications by enabling malicious script injection, compromising user data, and facilitating unauthorized actions. Traditional detection methods often struggle with evolving attack patterns. Machine learning offers a dynamic solution, effectively identifying both known and emerging XSS threats, enhancing cybersecurity, and protecting sensitive data.

### **4. How did you formulate the problem?**

This problem was formulated by recognizing the challenges in existing XSS detection mechanisms and the opportunity to address them using advanced machine learning models. The problem was structured as a binary classification task, it uses labeled benign and malicious code samples to develop an accurate, scalable, and adaptable detection system for evolving threats in web applications.

### **5. Which ML task you used?**

The machine learning task employed in this project is classification. The goal is to develop models that classify web-based code snippets into two categories:

- a) Benign (non-malicious): Representing safe code inputs.
- b) Malicious (XSS): Identifying potentially harmful code containing XSS vulnerabilities

## **DATASET**

### **1. Describe the dataset you have used**

The dataset is a Cross-Site Scripting (XSS) Dataset sourced from Kaggle, containing a collection of benign and malicious code snippets aimed at training models to detect XSS vulnerabilities.

## 2. Modality, sample size, features, and labels

- **Modality:** Text-based (HTML and JavaScript snippets)
- **Size:** Approximately 100,000 samples
- **Features:** Web script patterns such as `<script>` tags and `alert()` functions
- **Labels:** Binary (0 for benign code, 1 for XSS code)

## 3. How is the dataset collected?

Collected from real-world vulnerabilities and synthetic data representing XSS attack patterns. This is the link to access the dataset: [Cross-Site Scripting \(XSS\) Dataset](#).

## 4. Why is this dataset important?

XSS vulnerabilities are critical threats to web applications, making this dataset essential for enhancing web security. Its size and diversity capture both known and emerging attack patterns, supporting the creation of robust machine learning models. With labeled examples, the dataset enables effective training and evaluation of classification models, driving the development of practical solutions to detect and mitigate XSS vulnerabilities.

# MACHINE LEARNING METHODOLOGY

## 1. What are the methods used in the project?

The methodologies employed for this project are:

- **CNNs:** Extract hierarchical patterns and spatial features from the code snippets, such as identifying specific HTML and JavaScript tags indicative of XSS vulnerabilities.
- **RNNs:** Capture sequential dependencies in the web scripts, identifying patterns spread across multiple tokens that suggest complex XSS attacks.
- **Transformers:** Utilize self-attention mechanisms to analyze long-range dependencies and contextual relationships, making them adept at detecting novel and hidden XSS attack patterns.

Each method is used to capture different aspects of the data to detect Cross-Site Scripting (XSS) vulnerabilities:

## 2. Data Usage in this project

The dataset consisted of approximately 100,000 web-based code snippets, categorized as either benign or malicious (XSS). It was sourced from Kaggle.

- **Data Splits:** The data was split into training (70), validation (15), and testing (15) sets to ensure proper model evaluation and avoid overfitting.
- **Preprocessing Steps:**
  - **Tokenization and Embedding:** Code snippets were tokenized into smaller components, and embeddings were generated to represent them numerically for machine learning algorithms.
  - **Feature Engineering:** Features like HTML tags (`<script>`), JavaScript patterns (e.g., `alert()`), and other potential XSS indicators were highlighted.
  - **Padding and Normalization:** Scripts were padded to a fixed length to maintain uniform input sizes for CNNs and RNNs.
  - **Cleaning:** Duplicate entries were removed, and only relevant parts of the scripts were retained to reduce noise.
- **Handling Missing or Dirty Data:** The dataset did not contain missing values, but redundant entries were removed to ensure unbiased training. Irrelevant HTML tags and elements were stripped to focus on features essential to the model's learning objectives.

### 3. ML Frameworks and Libraries Used

The project was implemented using Python and the following libraries:

- **TensorFlow/Keras:** For building and training the CNN, RNN, and Transformer models.
- **NumPy and Pandas:** For data manipulation and preprocessing.
- **Scikit-learn:** For splitting datasets and evaluating models.
- **Matplotlib/Seaborn:** For visualizing results, including confusion matrices and performance metrics.

## RESULTS

### 1. Overview of the Algorithms:

- **CNN:**
  - **Key Features:** Convolutional layers for spatial pattern recognition, making them suitable for identifying local patterns within code snippets.
  - **Layers:** Convolutional layers combined with MaxPooling and 4 Dense layers for feature extraction and classification.
  - **Training Parameters:**
    - \* **Batch Size:** 128
    - \* **Epoch:** 10
- **RNN:**
  - **Key Features:** Bidirectional SimpleRNN layers for sequential data understanding, enabling the model to capture dependencies across time steps within code.
  - **Layers:** 3 Bidirectional SimpleRNN layers followed by Dense layers for feature processing and classification.
  - **Training Parameters:**
    - \* **Batch Size:** 128
    - \* **Epoch:** 10
- **Transformer:**
  - **Key Features:** Self-attention mechanism for capturing long-range dependencies, allowing the model to understand contextual relationships within code sequences.
  - **Layers:** 3 Transformer Encoder layers for advanced feature extraction and classification.
  - **Training Parameters:**
    - \* **Batch Size:** 128
    - \* **Epoch:** 10
    - \* **Embedding Size:** 64

### 2. Describe the results

The results for each model are summarized below, showcasing their performance across multiple met-rics:

| Model       | Accuracy | Validation Accuracy | Loss     | Validation Loss | Precision | Recall   |
|-------------|----------|---------------------|----------|-----------------|-----------|----------|
| CNN         | 0.992145 | 0.988313            | 0.025594 | 0.036848        | 0.987197  | 0.991204 |
| RNN         | 0.989041 | 0.989041            | 0.031572 | 0.038392        | 0.990074  | 0.990074 |
| Transformer | 0.998350 | 0.997800            | 0.004527 | 0.007571        | 0.990308  | 0.990308 |

Table 1: Performance metrics for CNN, RNN, and Transformer models.

### 3. Figures and Analysis:

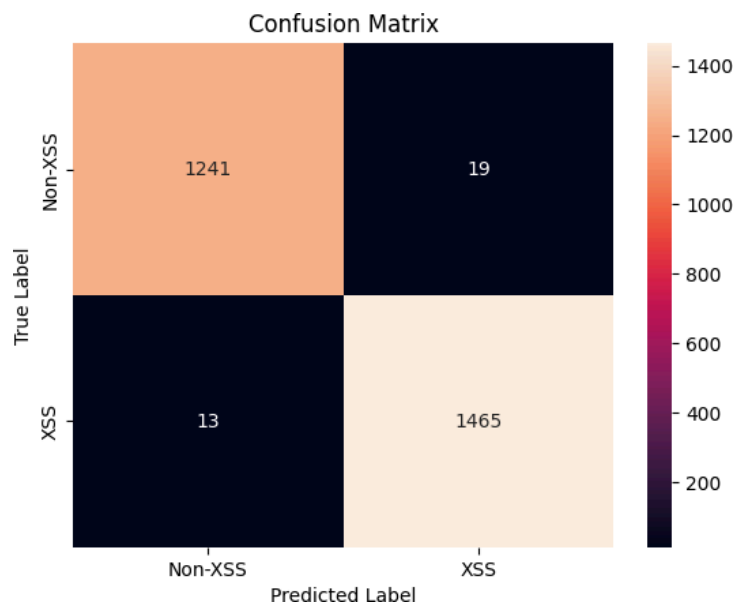


Figure 1: Confusion Matrix for the CNN Model

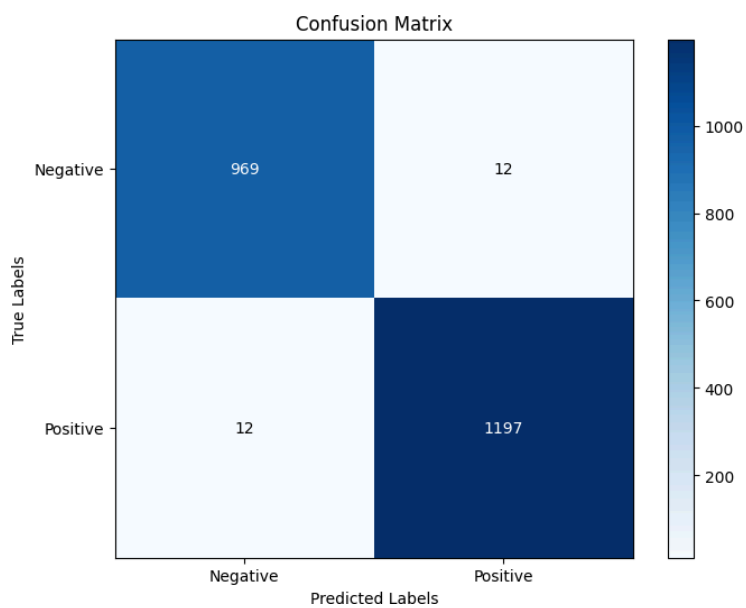


Figure 2: Confusion Matrix for the RNN Model

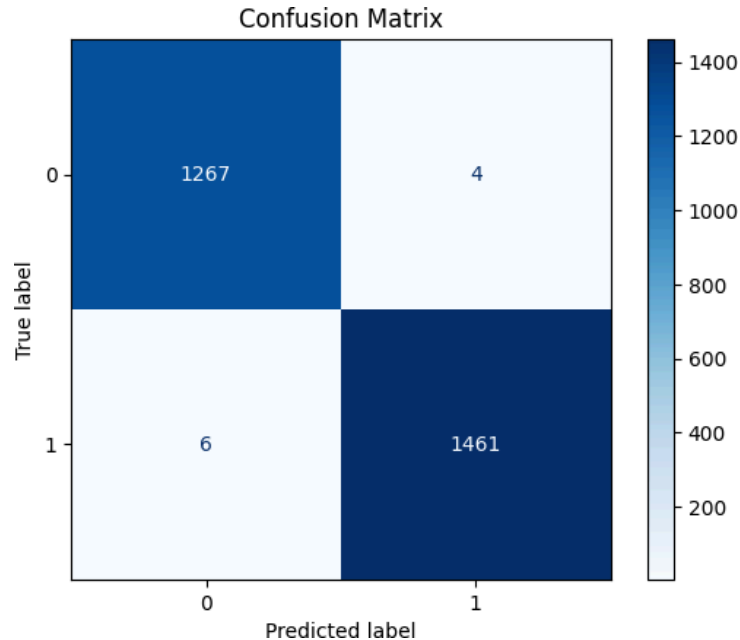


Figure 3: Confusion Matrix for the Transformer Model

#### 4. Comment on the best-performing algorithm

- **Best-Performing Model:** The **Transformer Model** achieved the highest accuracy (0.9983) and lowest loss (0.004527), making it the most effective model for XSS detection.
- **Reasons for Performance:**
  - Transformers use self-attention mechanisms, enabling them to understand long-range dependencies in code snippets, crucial for detecting sophisticated XSS attacks.
  - Transformers' ability to capture context across entire sequences gave them an edge over CNNs and RNNs, which primarily focus on local patterns or sequential dependencies, respectively.
- **Computational Complexity:**
  - Transformers, while highly effective, are computationally intensive due to their self-attention mechanism, which scales quadratically with input size.
  - CNNs and RNNs are less resource-demanding but trade off some accuracy and generalization capabilities.

#### 5. Are there any tradeoffs?

- **Tradeoffs Between Accuracy and Complexity:**
  - While Transformers provide superior accuracy, they require more computational power and memory, making them less suitable for resource-constrained environments.
  - CNNs, with simpler architectures, are faster to train and deploy but slightly underperform compared to Transformers in this task.
  - RNNs balance between the two, offering reasonable performance with moderate computational demands.
- **Tradeoffs Between Precision and Recall:**
  - CNNs achieved slightly higher recall than Transformers, making them slightly better at detecting all possible XSS threats. However, their lower accuracy suggests a higher false positive rate compared to Transformers.

Summary: For applications where accuracy is paramount, the Transformer model is the best choice due to its superior ability to capture complex dependencies, despite higher computational demands. CNNs and RNNs offer faster, more resource-efficient alternatives, providing solid performance for scenarios where speed and resource constraints are priorities, albeit with slightly lower accuracy.

## LESSONS LEARNED

### 1. What did you learn?

Through this project, I gained valuable insights into the practical application of machine learning models for cybersecurity, specifically in detecting Cross-Site Scripting (XSS) vulnerabilities. I learned the importance of feature selection and data preprocessing for effective model training, and how different models, from CNNs to Transformers, can be tailored for specific tasks. Additionally, I developed a deeper understanding of the limitations and strengths of each model type, learning that more complex models like Transformers, while more accurate, come with increased computational demands.

### 2. What was important in this problem?

The most important aspect of this problem was understanding the dataset and using it effectively to train accurate models. Proper data preprocessing, such as tokenization and feature extraction, was essential to ensure that the models could learn from the data efficiently. Choosing the right model architecture and fine-tuning the hyperparameters were also critical steps that directly influenced the outcomes.

Another key consideration was recognizing the trade-offs between model complexity, accuracy, and computational requirements. Simpler models, while faster and easier to train, may not reach the same level of accuracy as more complex models. On the other hand, more complex models, like transformers, offered better performance but required significantly more computational resources and longer training times.

### 3. What were the challenges?

I faced several challenges during the project:

- **Data Preprocessing:** Ensuring that the dataset was in the proper format for training and removing any noise that could affect model performance. This involved tasks such as cleaning HTML tags and JavaScript elements while maintaining important contextual information. This step was time-intensive and required precise handling to avoid losing critical features.
- **Model Selection and Tuning:** Choosing the right model architecture and hyperparameters was challenging, as it involved experimentation and fine-tuning. For example, tuning the Transformer model's number of layers and attention heads to achieve optimal performance was intricate.
- **Computational Resources:** Training the Transformer model required significant computational power and memory, which posed limitations in terms of training time and resource availability.
- **Balancing Precision and Recall:** Achieving a balance between precision and recall was difficult during model optimization. While high recall was essential for capturing as many XSS instances as possible, it sometimes came at the cost of precision, leading to an increased number of false positives.

### 4. What should the reader get out of reading your project?

Readers should gain an understanding of how machine learning can be applied to enhance cybersecurity, particularly in the detection of Cross-Site Scripting (XSS) vulnerabilities. This project highlights how deep learning models, such as CNNs, RNNs, and Transformers, can be utilized to recognize patterns and understand context within web code, effectively distinguishing between benign and malicious code.

This project highlights how important data preprocessing and feature engineering are for building effective machine learning models. Steps like tokenization, feature extraction, and data cleaning help the model learn better and make accurate predictions.

Lastly, readers should understand that while complex models like Transformers offer higher accuracy, simpler models like CNNs and RNNs can be more practical in scenarios with limited computational resources. The lessons learned here emphasize the importance of carefully assessing the problem requirements, computational constraints, and desired outcomes to select the most suitable machine learning approach.

